

Chapter 16

Application: Data Driven Pages

1. Introduction

Data Driven Pages (DDP) are a series of layout pages created from different partitions of a map in a map document. The extent of each page is defined (driven) by a feature in a feature layer called index layer in the same data frame, and the layout of all pages are defined by one layout design (Figure 1). DDP allow you to build a map book and to export a PDF document.

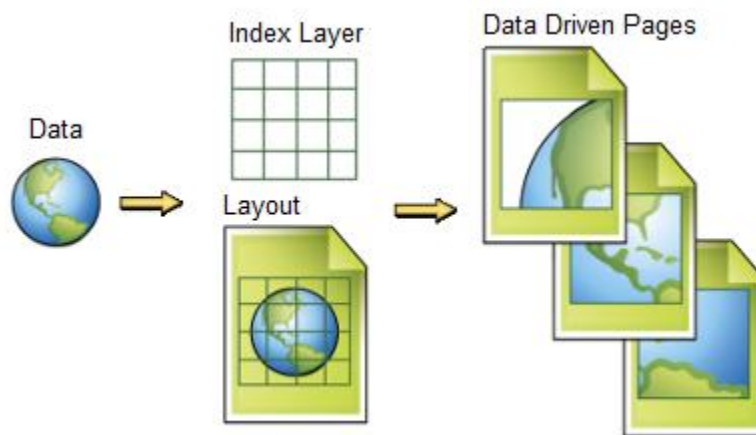


Figure 1 Data Driven Pages

To create a series of Data Driven Pages requires 1) a detailed data frame containing the data you want to map; 2) an index layer (feature class) containing features that can be used to define extent of each layout page. The index layer usually needs to have a Name field that holds a name of each layout page as well as a Sort field (usually a page number field) that can be used to sort output pages. The index layer can carry other fields, such as rotation and spatial reference, to define more properties of the pages; and 3) a layout design that defines the layout of all output pages. A layout page contains a data frame and may include other map elements such as a title, a page name, a page number, a scale bar, a north arrow, etc.

An index layer can be a polygon layer, polyline layer, or point layer. With a polygon or polyline layer, the geographic extent of each feature defines the extent of a layout page. If the index layer is a point layer, each point feature defines the center of a layout page and a scale field usually needs to be used to define the map scale of each page. If a scale field is not available, the current map scale of the main data frame can be used for every page.

ArcMap has a Data Driven Pages toolbar. Once you load a map along with an index layer in a data frame in ArcMap, you can set up DDP by clicking the “Data Driven Pages Setup” button on the toolbar to open the DDP setup dialog box. Then you can check the “Enable Data Driven Pages” check box and configure necessary parameters in the dialog box (Figure 2). To learn

how to create DDP, please refer to “Creating Data Driven Pages” web page at the ArcGIS Resources website¹.

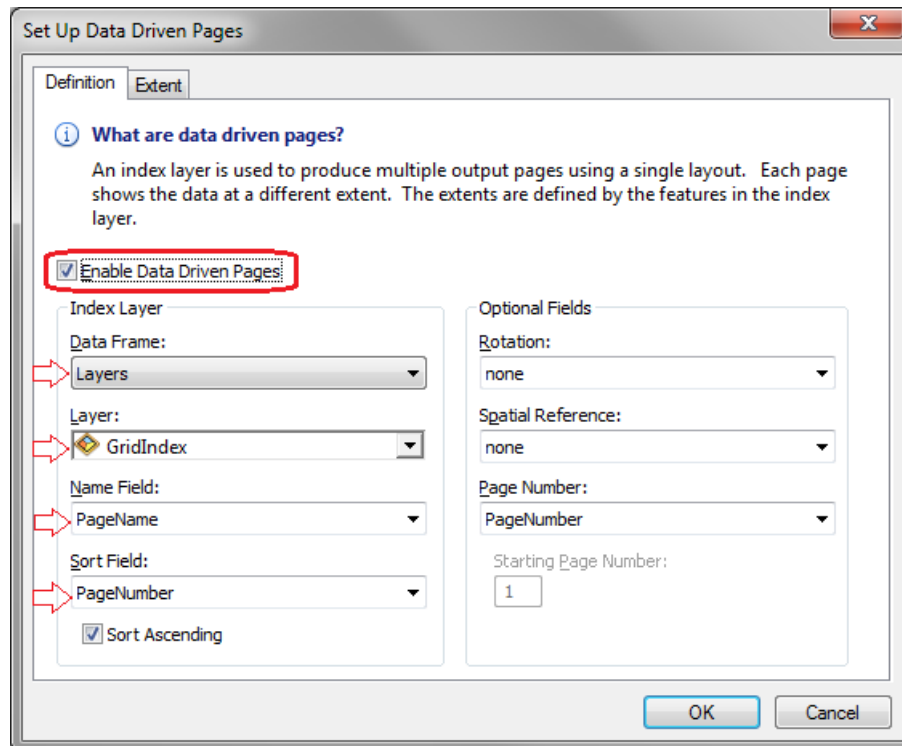


Figure 2 Data Driven Pages set up dialog box

To create data driven pages involves a few tasks including to create an index layer, to design a page layout, and to set up DDP, each involving multiple steps. For example, to prepare a grid index layer with necessary fields in the attribute table may be accomplished by using several geoprocessing tools: 1) use the “Grid Index Features” tool create a polygon grid feature class. With this tool, the user needs to specify the origin coordinates of the feature class and number of rows and columns of the grid in a standard geoprocessing dialog box; 2) use the “Calculate Adjacent Fields” tool to create and populate eight fields that define neighbors of each quadrat in the grid; 3) use the “Calculate UTM Zone” tool to create a field that specifies UTM zones for the pages defined by each quadrat in the grid.

This chapter will instruct you to create a custom add-in tool that automates the process of creating a grid index layer, creating a page layout, and setting up DDP. To create a grid index layer, the tool will allow the user to interactively define and adjust a map extent and grid size of the index layer. The add-in will also provide a dockable windows for the user to modify parameters including the precise map extent, number of rows and columns of grids, as well as necessary fields.

The purpose of this chapter is not to create a perfect DDP product but to use the process as a comprehensive example to demonstrate ArcGIS desktop add-in and ArcObjects programming capabilities and techniques. Several new add-in features and programming techniques will be introduced, which include using add-in extension, creating custom events, designing page layout, creating map surrounds, etc.

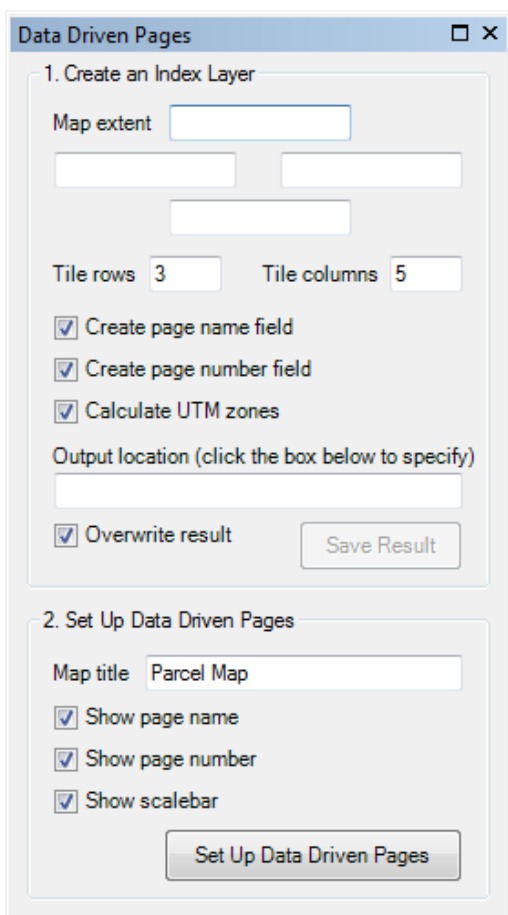
¹ <http://resources.arcgis.com/en/help/main/10.2/index.html#//00s90000003n000000>.

2. Data Driven Pages Add-in Project

2.1 Starting a new ArcMap add-in project

Please start a new ArcMap add-in project named “DataDrivenPages”. Then create a dockable window, an add-in tool, and two buttons in the project. Use a toolbar to hold the add-in tool and buttons. One of the add-in buttons will be used to launch the dockable window, the add-in tool will be used to draw a rectangle on the map to define a map extent, and the other add-in button will be used to display the built-in Data Driven Pages toolbar. Please refer to Figure 3 to create the key add-in elements.

Add-in	Name	Image	Caption and Tip
Button	ShowWindowButton		Show dockable window
Tool	MappingExtentTool		Mapping extent
Button	ShowDDPTlbrButton		Show Built-in DDP toolbar
DockableWindow	DockableWindow		Caption: Data Driven Pages
Toolbar			My DDP Toolbar



Design of the Dockable Window

Control	Property Settings
GroupBox	Text: 1. Create Index Layer
GroupBox	Text: 2. Create Data Driven Pages
TextBox	Name: txtExtTop
TextBox	Name: txtExtLeft
TextBox	Name: txtExtRight
TextBox	Name: txtExtBottom
TextBox	Name: txtTileRows
TextBox	Name: txtTileCols
CheckBox	Name: chkCreatePageName
CheckBox	Name: chkCreatePateNumber
CheckBox	Name: chkCalcUTMZones
TextBox	Name: txtOutput
CheckBox	Name: chkOverwrite
Button	Name: btnSave; Enabled: False; Text: Save Result
TextBox	Name: txtMapTitle
CheckBox	Name: chkShowPageName
CheckBox	Name: chkShowPageNumber
CheckBox	Name: chkShowScalebar
Button	Name: btnSetupDDP Text: Set Up Data Drive Pages

Figure 3 GUI design of the Data Driven Pages project

We hope that when the user uses the map extent add-in tool to drag a rectangle in the map area, a rectangle element containing the specified rows and columns of rectangle elements (tiles) are added into the map, and the mapping extent coordinates are displayed in the first four text boxes. When the overall rectangle element is selected, the user can resize it by dragging the handles around it, which will cause all tiles to be automatically resized and coordinates in the dockable window to be updated (Figure 4). On the other hand, when the user edits values in the text boxes, the rectangle element and tiles inside it will also be updated. Moreover, the program should allow the user to delete unnecessary tiles.

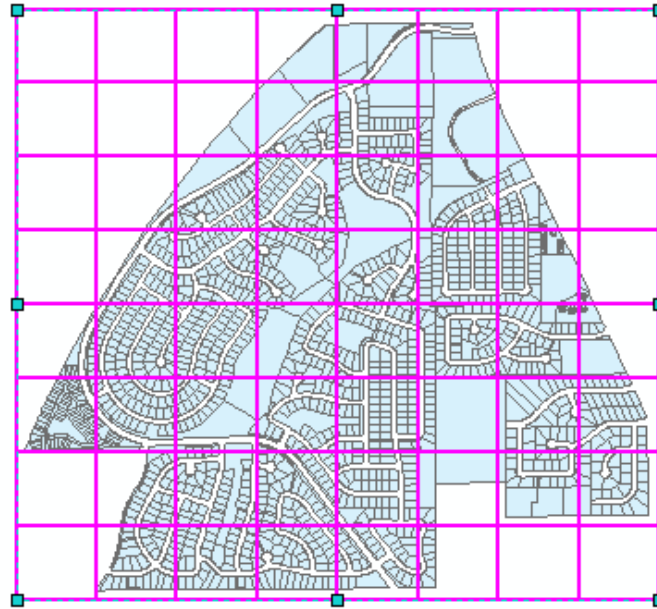


Figure 4 Mapping extent and tile elements

In addition, when the user clicks on the output location box, a dialog box will pop up for the user to browse a file or personal geodatabase and specify a feature class name, and the Save Result button will be enabled. When the user clicks on the Save Result button, a new grid index feature class will be created in the specified location and automatically loaded into the map. The index feature class should include user specified fields which are also populated with proper values. The overwrite check box will allow new index feature class to overwrite an existing one if checked.

Once a map and an index layer are loaded or created, the user can click the “Set Up Data Driven Pages” button to set up data driven pages. This button will first create a page layout by properly sizing and locating the map frame on the layout page, creating a static text element for a map title, as well as inserting a few dynamic elements such as a page name, a scale bar, and a page number based on user selection. It then displays the built-in Data Driven Pages toolbar and opens the built-in Data Driven Pages set up dialog box (Figure 2).

Before implementing functionalities of the add-in project, please add the following .NET assemblies into the project as references:

ESRI.ArcGIS.ADF.Local – required for creating font object
 ESRI.ArcGIS.ArcMap

```

ESRI.ArcGIS.Carto
ESRI.ArcGIS.Catalog
ESRI.ArcGIS.CatalogUI
ESRI.ArcGIS.DataSourcesFile
ESRI.ArcGIS.DataSourcesGDB
ESRI.ArcGIS.Display
ESRI.ArcGIS.Geodatabase
ESRI.ArcGIS.Geometry
stdole - required for creating text font. It is a Microsoft .NET assembly

```

2.2 Launching the dockable window

Launching a dockable window is introduced in Section 3.1 of Chapter 11. To implement the add-in button “ShowWindowButton”, please add the following namespace importing statements into the class module of the button.

```

Imports ESRI.ArcGIS.Framework ' IApplication, IDockableWindowManager, IDockableWindow
Imports ESRI.ArcGIS.esriSystem ' needed by the UID class

```

Then implement the OnClick event procedure as the following.

```

Protected Overrides Sub OnClick()
    Dim app As IApplication = My.ArcMap.Application ' gets the application object
    Dim dockWindowManager As IDockableWindowManager =
        CType(app, IDockableWindowManager) ' QI
    Dim WinUID As UID = New UID ' instantiates UID object
    WinUID.Value = My.ThisAddIn.IDs.DockableWindow ' sets Dockable Window GUID
    Dim dockwin As IDockableWindow = dockWindowManager.GetDockableWindow(WinUID)
    If dockwin Is Nothing Then Return

    ' show/hide the dockable window
    If dockwin.IsVisible Then
        dockwin.Show(False)
    Else
        dockwin.Show(True)
    End If

    My.ArcMap.Application.CurrentTool = Nothing ' unselect any tool
End Sub

```

2.3 Showing built-in Data Driven Pages toolbar

To obtain an ArcGIS built-in command such as a tool, button, or menu item, or a toolbar is introduced in a box in Section 3.1 of Chapter 11. To access a built-in command, you must find the GUID value of the command at the ArcObjects Help for .NET website², and then call the Find() method of the CommandBars object. If the command is a tool, you can execute it by calling its Execute() method; if the command is a toolbar, you can display it by calling its Dock() method.

To show the built-in Data Driven Pages toolbar, please add the following namespace importing statements into the class module of the ShowDDPTlbrButton add-in button.

² <http://resources.arcgis.com/en/help/arcobjects-net/conceptualhelp/index.html#/00010000029s000000>

```
Imports ESRI.ArcGIS.Framework ' needed by ICommandBar
Imports ESRI.ArcGIS.esriSystem ' needed by UID class
Imports ESRI.ArcGIS.ArcMapUI ' needed by IDocument
```

Since the GUID of the built-in Data Driven Pages toolbar is "{EF545F90-9F94-462E-8692-035923C87193}", you can implement the OnClick event procedure as the following.

```
Protected Overrides Sub OnClick()
    ' display the built-in DDP toolbar
    Dim doc As IDocument = My.ArcMap.Application.Document
    Dim ddpUID As UID = New UID()
    ddpUID.Value = "{EF545F90-9F94-462E-8692-035923C87193}"
    Dim ddpToolbar As ICommandBar = doc.CommandBars.Find(ddpUID)
    ddpToolbar.Dock(esriDockFlags.esriDockShow)
End Sub
```

3. Creating An Extension

An ArcGIS Desktop add-in project can include a special class called Extension. An Extension class has an OnStartup event and an OnShutdown event that react to the starting and closing of ArcGIS Desktop application (e.g. ArcMap). Extension is commonly used to coordinate activities between other add-in components such as buttons, tools and dockable windows within a project. In this project, for example, we can use an Extension class to maintain a set of shared variables to keep track of the bounds of the mapping extent as well as the number of rows and columns of tiles. We can also declare a shared Boolean variable in the Extension class to keep track of the open status of the dockable window, so that the mapping extent tool knows whether the dockable window is loaded or not when it tries to write values into text boxes in the window. Moreover, we can create custom events in the Extension class to carry out tasks.

You can create an Extension class using the add-in wizard like creating an add-in button or tool. Please do the following to add an extension into the current project.

- 1) Right click on the project title in the Solution Explorer and select Add >> New Item...
- 2) In the Add New Item dialog box, select ArcGIS >> Desktop Add-ins >> Add-in Component, provide a name "Extension" in the item name box, and click Add
- 3) In the ArcGIS Add-Ins Wizard dialog box, select "Extension" in the add-in component list, check "Auto-load" and "Show in Extension Manager", and enter a product name "Data Driven Pages" and some descriptive text. Click the Finish button to complete.

An Extension module with name "Extension.vb" will be created in the Solution Explorer and a new element will be added in the XML configuration document. Please double click the module in the Solution Explorer to open it in the code editor, then add the following shared member variables inside the class just below the inherit statement.

```
Friend Shared WindowLoaded As Boolean = False
Friend Shared ExtentTop As Double
Friend Shared ExtentBottom As Double
Friend Shared ExtentLeft As Double
Friend Shared ExtentRight As Double
Friend Shared TileRows As Integer = 3
```

```
Friend Shared TileCols As Integer = 5
```

Since these variables are declared with keywords “Friend” and “Shared”, they can be accessed from any modules in the current project. A shared member of a class can be accessed without having to use an instance of the class. It can be accessed by referring to the class as a namespace. For example, you can update the text boxes in the Load event of the dockable window using value of the shared member variables of the Extension class as the following.

```
Private Sub DockableWindow_Load(sender As Object, e As EventArgs) Handles MyBase.Load
    Extension.WindowLoaded = True ' updates var WindowLoaded in extension

    ' gets initial values from member variables of the Extension class
    txtExtTop.Text = Extension.ExtentTop
    txtExtBottom.Text = Extension.ExtentBottom
    txtExtLeft.Text = Extension.ExtentLeft
    txtExtRight.Text = Extension.ExtentRight
    txtTileRows.Text = Extension.TileRows
    txtTileCols.Text = Extension.TileCols
End Sub
```

On the other hand, when any of the four text boxes for mapping extent in the dockable window is modified by the user, you can update the four mapping extent variables in the extension class using the TextChanged events of the text boxes. This task could be accomplished using the TextChanged event of each text box, but it can be done more elegantly using one procedure that handles four events as the following.

```
Private Sub ExtentBox_Changed(sender As Object, e As EventArgs) _
    Handles txtExtTop.TextChanged, txtExtBottom.TextChanged,
            txtExtLeft.TextChanged, txtExtRight.TextChanged
    ' parameter sender holds the (text box) object that fires the event,
    ' since the parameter is declared as a general object type,
    ' we must cast it to a windows text box control in order to use it
    Dim box As Windows.Forms.TextBox = TryCast(sender, Windows.Forms.TextBox)

    If Not IsNumeric(box.Text) Then Return ' if text in the box is not a number
    Dim value As Double = CDb1(box.Text) ' converts it to a double type value

    ' depending on the name of the text box that is fired
    ' assign the value to appropriate variable in the extension class
    Select Case box.Name
        Case "txtExtTop"
            Extension.ExtentTop = value
        Case "txtExtBottom"
            Extension.ExtentBottom = value
        Case "txtExtLeft"
            Extension.ExtentLeft = value
        Case "txtExtRight"
            Extension.ExtentRight = value
    End Select
End Sub
```

To handle multiple events by one procedure, the key is the list of events following keyword “Handles”. In the above event procedure, the TextChanged events of four text boxes are listed after keyword “Handles”, therefore if the content of any of the text boxes is updated, the procedure is invoked. To create the above procedure, you can bring up the blank TextChanged

event procedure of any of the text boxes, then modify the procedure name and add necessary TextChanged events in the handle list. The name only describes the procedure, and it does not determine capability of the event.

But how does the procedure know which text box triggers the event? Parameter *sender* holds the object that fires the procedure. Since this parameter is declared as a general object type, to use it as a text box (since you need to call its Text property), you must explicitly cast it into a text box object although, as the programmer, you know the object is a text box. Once you get the text box object, you can retrieve the text from it, convert the text into a double type value, and assign the value to an appropriate member variable of the Extension class.

Similarly, you can use the following procedure to update the tile row and column member variables of the Extension class when values in corresponding text boxes in the dockable window are updated.

```
Private Sub TileRowCol_TextChanged(sender As Object, e As EventArgs) Handles _
    txtTileRows.TextChanged, txtTileCols.TextChanged
    ' get the text box object
    Dim box As Windows.Forms.TextBox = TryCast(sender, Windows.Forms.TextBox)

    If Not IsNumeric(box.Text) Then Return ' validate the value in the text box
    Dim value As Integer = CInt(box.Text)
    If value <= 0 Then ' if the user enters a negative number or zero
        Windows.Forms.MessageBox.Show("Invalid number", "Error",
                                        Windows.Forms.MessageBoxButtons.OK,
                                        Windows.Forms.MessageBoxIcon.Error)
        ' restore the original value stored in the variable in Extension class
        If box.Name = "txtTileRows" Then
            box.Text = Extension.TileRows
        Else
            box.Text = Extension.TileCols
        End If
    Else ' if the value is a positive number
        If box.Name = "txtTileRows" Then ' store the value into a variable
            Extension.TileRows = value
        Else
            Extension.TileCols = value
        End If
    End If
End Sub
```

4. Creating Mapping Extent and Tiles

To implement the mapping extent tool, you can instantiate a rubber band object for the user to track an envelope which is then used to create a rectangle element to display a mapping extent. Please refer to Chapter 11 if you need help in using the rubber band object to track geometric shapes.

Once an envelope object is tracked, you can display its four bounds in the map extent boxes in the dockable window if the window is loaded. Once a value is assigned to a map extent box, the TextChanged event will be fired which updates a corresponding member variable in the Extension class. Otherwise, if the dockable window is not loaded, the four bounds of the

envelope is directly assigned to the member variables in the Extension class. Moreover, once an envelope object is tracked, you will also create tiles (rectangle elements) of specified number of rows and columns and display them in the map.

Since the text boxes in the dockable window are private controls that are not accessible to external classes, in order to update these text boxes from the mapping extent tool class, you must create a shared variable in the dockable window as a delegate of each text boxes. To do so, please declare the following shared variables in the DockableWindow class just below the class declaration line.

```
Friend Shared TextBox_ExtTop As Windows.Forms.TextBox
Friend Shared TextBox_ExtBottom As Windows.Forms.TextBox
Friend Shared TextBox_ExtLeft As Windows.Forms.TextBox
Friend Shared TextBox_ExtRight As Windows.Forms.TextBox
Friend Shared TextBox_TileRows As Windows.Forms.TextBox
Friend Shared TextBox_TileCols As Windows.Forms.TextBox
```

Then add six lines of code into the Load event of the dockable window to link each shared variable (delegate) to an appropriate text box object.

```
TextBox_ExtTop = txtExtTop           ' links a text box object to a variable
TextBox_ExtBottom = txtExtBottom     ' same as above
TextBox_ExtLeft = txtExtLeft         ' same as above
TextBox_ExtRight = txtExtRight       ' same as above
TextBox_TileRows = txtTileRows       ' same as above
TextBox_TileCols = txtTileCols       ' same as above
```

After making the above treatment to the dockable window, the mapping extent tool class can access these text boxes through their shared delegates. Now, you can begin to implement functionalities of the mapping extent tool. Please add the following namespace importing statements in the tool's module.

```
Imports ESRI.ArcGIS.Framework
Imports ESRI.ArcGIS.ArcMapUI
Imports ESRI.ArcGIS.Carto
Imports ESRI.ArcGIS.Display
Imports ESRI.ArcGIS.Geometry
```

The task is carried out in four major steps:

- 1) Create a rubber band to track an envelope object
- 2) Second, update the map bound text boxes in the dockable windows if the window is open, which will trigger the TextChanged event of the text boxes to update the extent member variables of the Extension class. Otherwise, directly assign map bounds to the extent member variables in the Extension class.
- 3) Create a matrix of rectangle elements (tiles) within the mapping extent based on user specified rows and numbers in the row/column text boxes.
- 4) Create an overall rectangle element for the mapping extent and make it invisible. These steps are implemented in the OnMouseDown event of the mapping extent tool as the following.

```
Protected Overrides Sub OnMouseDown(...)
    Dim mxDoc As IMxDocument = My.ArcMap.Document ' gets the document
    Dim gc As IGraphicsContainer = mxDoc.FocusMap ' gets the graphics container
```

```

Dim av As IActiveView = mxdDoc.ActiveView          ' gets the active view

' to track a mapping extent, make sure the active view is not page layout
If TypeOf av Is IPageLayout Then Return

' 1. use a rubber band object to track a rectangle
Dim rubberBand As IRubberBand = New RubberEnvelope()
Dim scrnDisp As IScreenDisplay = av.ScreenDisplay
Dim shape As IGeometry = rubberBand.TrackNew(scrnDisp, Nothing)
If shape Is Nothing Then Return
If shape.IsEmpty Then Return

' 2. if the dockable windows is loaded, update the text boxes in the window
' otherwise, directly store extent bounds into Extension member variables
Dim rect As IEnvelope = shape ' QI
If Extension.WindowLoaded Then
    DockableWindow.TextBox_ExtTop.Text = Math.Round(rect.YMax, 6)
    DockableWindow.TextBox_ExtBottom.Text = Math.Round(rect.YMin, 6)
    DockableWindow.TextBox_ExtLeft.Text = Math.Round(rect.XMin, 6)
    DockableWindow.TextBox_ExtRight.Text = Math.Round(rect.XMax, 6)
    ' these will fire TextChanged events, which update Extension members
Else
    Extension.ExtentTop = Math.Round(rect.YMax, 6)
    Extension.ExtentBottom = Math.Round(rect.YMin, 6)
    Extension.ExtentLeft = Math.Round(rect.XMin, 6)
    Extension.ExtentRight = Math.Round(rect.XMax, 6)
End If

' 3. create a matrix of tiles
'first, delete all existing elements in the map
gc.DeleteAllElements()
' then call a custom function (to be created) to create tiles
CreateTiles(rect, Extension.TileRows, Extension.TileCols)

' 4. create an invisible rectangle element for the overall mapping extent
' this element will allow the user to resize the mapping extent
' 4.1 create a new rectangle element object
Dim rectElement As IRectangleElement = New RectangleElement()
Dim element As IElement = rectElement ' QI
element.Geometry = shape ' sets the geometry property
Dim elementProps As IElementProperties = element ' QI
elementProps.Name = "Mapping Extent" ' assigns a name to the element

' 4.2 create a simple line symbol for the border
Dim lineSym As ISimpleLineSymbol = New SimpleLineSymbol()
lineSym.Color = CreateColor(255, 0, 255, True) ' line is transparent (invisible)
lineSym.Width = 2.0

' 4.3 creates a simple fill symbol object
Dim sfSymbol As ISimpleFillSymbol = New SimpleFillSymbol()
sfSymbol.Outline = lineSym
sfSymbol.Color = CreateColor(255, 255, 0, True) ' transparent fill color

' 4.4 sets the symbol property
Dim fillElement As IFillShapeElement = element ' QI
fillElement.Symbol = sfSymbol

gc.AddElement(element, 0) ' adds the extent element on the top

```

```

av.Refresh()

' after tracking a mapping extent, select the ArcMap built-in select tool
My.ArcMap.Application.CurrentTool = GetCommand("esriArcMapUI.SelectTool")
MyBase.OnMouseDown(arg)
End Sub

```

This event procedure creates an invisible rectangle element and assigns a name “Mapping Extent” to it. This invisible element will allow the user to select and resize so as to update the mapping extent. The procedure calls three custom functions to perform specific tasks: CreateTiles(), CreateColor(), and GetCommand(). Function CreateColor() takes three values for red, green, and blue, with an optional parameter to indicate transparency. It returns an RGB color object.

```

Private Shared Function CreateColor(ByVal Red As Integer,
                                   ByVal Green As Integer,
                                   ByVal Blue As Integer,
                                   Optional ByVal Transparent As Boolean = False) As IRgbColor
    Dim color As IRgbColor = New RgbColor()
    With color
        .Red = Red
        .Green = Green
        .Blue = Blue
        If Transparent Then
            .Transparency = 0
        Else
            .Transparency = 255
        End If
    End With
    Return color
End Function

```

When the Transparency property of an RgbColor object is set to 0 the color is transparent; when the property is set with any value between 1 and 255, the color is opaque.

Function GetCommand() takes a GUID as input and returns an ArcGIS built-in command object. A command object can be a tool, a button, a menu item, or a toolbar.

```

Private Function GetCommand(GUID As String) As ICommandItem
    Dim cmdUID As ESRI.ArcGIS.esriSystem.UID = New ESRI.ArcGIS.esriSystem.UID
    cmdUID.Value = GUID
    Dim cmdBars As ICommandBars = My.ArcMap.Application.Document.CommandBars
    Dim cmdItem As ICommandItem = cmdBars.Find(cmdUID)
    Return cmdItem
End Function

```

Sub procedure CreateTiles() takes an envelope object, a row number, and column number as input. It creates a matrix of rectangle elements and adds the elements into the map to display. The function is declared with keywords “Friend Shared” so that it can be called from other classes in the project when needed. In particular, this procedure will also be called in the dockable window class when the mapping extent or number of tile rows or columns is changed. The procedure will also be called by a custom event in the Extension class that reacts to changes occurring to the overall mapping extent element.

```

Friend Shared Sub CreateTiles(extent As IEnvelope, rows As Integer, cols As Integer)

```

```

' get coordinates of the top left corner of the mapping extent
Dim x0 As Double = extent.XMin
Dim y0 As Double = extent.YMax
Dim tilewidth As Double = extent.Width / cols ' tile width
Dim tileheight As Double = extent.Height / rows ' tile height

DeleteTiles() ' calls a function to delete all existing tiles
Dim gc As IGraphicsContainer = My.ArcMap.Document.FocusMap ' graphics container

Dim rect As IPolygon = Nothing
Dim tile As IElement = Nothing
Dim name As String = ""
Dim zorder As Integer = 1
For i As Integer = 0 To rows - 1
    For j As Integer = 0 To cols - 1
        ' create a rectangle polygon
        rect = CreateRectangle(x0 + tilewidth * j,
                                y0 - tileheight * i,
                                tilewidth,
                                tileheight)
        name = "Tile: " & Chr(i + 65) & (j + 1).ToString ' name a tile
        tile = CreateTileElement(rect, name) ' creates an element
        gc.AddElement(tile, zorder) ' adds element into map
        zorder += 1
    Next
Next
BringExtentBoxToTop() ' brings the overall mapping extent element to top
Dim av As IActiveView = gc
av.Refresh()
End Sub

```

Tiles are created by calling two custom functions, `CreateRectangle()` and `CreateTileElement()`, inside the nested loops. Once a tile element is created, it is assigned a name by code

```
name = "Tile: " & Chr(i + 65) & (j + 1).ToString
```

The resulted tile name will have a format of “Tile: A#” where “A” represents a row letter and “#” represents a column number. For example “Tile: B3” is located in the second row and the third column. For the first row, since *i* is 0, `Chr(0 + 65)` is letter “A”.

Moreover, procedure `CreateTiles()` calls other four procedures to perform sub-tasks: sub procedure `DeleteTiles()`, function `CreateRectangle()`, function `CreateTileElement()`, and sub procedure `BringExtentBoxToTop()`. Procedure `DeleteTiles()` examines each element in the graphics container (i.e. the map). If an element is named starting with “Tile:”, then it is deleted.

```

Private Shared Sub DeleteTiles()
    Dim av As IActiveView = My.ArcMap.Document.ActiveView
    Dim gc As IGraphicsContainer = av
    gc.Reset()

    Dim elem As IElement = gc.Next() ' gets the first element
    Dim props As IElementProperties
    While elem IsNot Nothing
        props = elem ' QI to the IElementProperties interface
        If props.Name.Substring(0, 5) = "Tile:" Then
            gc.DeleteElement(elem)
        End If
        elem = gc.Next()
    End While
End Sub

```

```

        End If
        elem = gc.Next()      ' gets the next element
    End While
End Sub

```

Function `CreatRectangle()` takes four arguments, left, top, width, and height and returns a polygon object (IPolygon interface). The code for this function is quite straightforward.

```

Private Shared Function CreateRectangle(left As Double,
                                       top As Double,
                                       width As Double,
                                       height As Double) As IPolygon
    Dim poly As IPolygon = New Polygon      ' instantiates a polygon object
    Dim pcoll As IPointCollection = poly    ' QI to the IPointCollection interface

    ' create four point objects for the four corners
    Dim p1 As IPoint = New Point
    Dim p2 As IPoint = New Point
    Dim p3 As IPoint = New Point
    Dim p4 As IPoint = New Point
    p1.PutCoords(left, top)                 ' top left corner
    p2.PutCoords(left + width, top)         ' top right corner
    p3.PutCoords(left + width, top - height) ' bottom right corner
    p4.PutCoords(left, top - height)        ' bottom left corner

    ' sequentially add the points into the polygon (IPointCollection interface)
    With pcoll
        .AddPoint(p1)
        .AddPoint(p2)
        .AddPoint(p3)
        .AddPoint(p4)
        .AddPoint(p1)
    End With
    Return poly      ' returns the polygon object
End Function

```

The `CreateTileElement()` function takes two arguments, a polygon object and a name, and it returns a polygon element object. You should be familiar with the code of this function.

```

Private Shared Function CreateTileElement(poly As IPolygon,
                                         name As String) As IElement
    Dim polyElement As IPolygonElement = New PolygonElement()
    Dim fsElement As IFillShapeElement = polyElement

    Dim element As IElement = polyElement
    element.Geometry = poly

    Dim props As IElementProperties = element
    props.Name = name

    Dim sfSymbol As ISimpleFillSymbol = New SimpleFillSymbol()
    sfSymbol.Color = CreateColor(255, 255, 255, True) ' transparent fill color
    Dim slnSymbol As ISimpleLineSymbol = New SimpleLineSymbol()
    slnSymbol.Color = CreateColor(255, 0, 255)        ' magenta border color
    slnSymbol.Width = 1.5
    sfSymbol.Outline = slnSymbol

```

```

        fsElement.Symbol = sfSymbol
    Return element
End Function

```

Once all the tile elements are created, the invisible element representing the overall mapping extent is covered by the tiles. In the end of the CreateTiles() procedure, you need to bring that element to top. To do so, you can iterate through all elements in the graphics container to find the one named "Mapping Extent", then delete it from the container and add it back on the top.

```

Private Shared Sub BringExtentBoxToTop()
    Dim av As IActiveView = My.ArcMap.Document.ActiveView
    Dim gc As IGraphicsContainer = av
    gc.Reset()
    Dim elem As IElement = gc.Next()
    Dim props As IElementProperties
    While elem IsNot Nothing
        props = elem
        If props.Name = "Mapping Extent" Then ' the element is found
            gc.DeleteElement(elem)
            gc.AddElement(elem, 0)
            Exit While ' exit the loop
        End If
        elem = gc.Next()
    End While
End Sub

```

After implementing the mapping extent tool, you can run the program and test the tool. Once you use the tool to drag a rectangle in the map area, the specified number of rows and columns of tiles will be created, and the overall mapping extent will display in the text boxes in the dockable window if opened. If everything works normally, you can proceed to the next step.

5. Updating Mapping Extent and Tiles

We hope this add-in supports resizing the graphic mapping extent and updating tiles in two ways: 1) by editing values in the mapping extent text boxes in the dockable window, and 2) by dragging handles of the invisible rectangle element that represents the overall mapping extent.

5.1 Editing extent coordinates

After the user edits the mapping extent coordinates or number of rows/columns in a text box, the KeyDown event of the text box is fired. Moreover, if the Enter key is pushed after editing the value in a text box, you can call a procedure to update the mapping extent element and tiles. The following event procedure handles the KeyDown event of six text boxes in the dockable window class.

```

Private Sub TextBox_KeyDown(sender As Object,
    e As System.Windows.Forms.KeyEventArgs) _
    Handles txtExtTop.KeyDown, txtExtBottom.KeyDown,
    txtExtLeft.KeyDown, txtExtRight.KeyDown,
    txtTileRows.KeyDown, txtTileCols.KeyDown
    If e.KeyCode = Windows.Forms.Keys.Enter Then ' the Enter key is pushed
        UpdateMappingExtent()
    End If
End Sub

```

```

End If
End Sub

```

Please note that when text in the mapping extent text boxes is changed, the extent variables in the Extension class are updated (by TextChanged event). Procedure UpdateMappingExtent() first validates the new values stored in extent variables in the Extension class, then it gets the overall mapping extent element from the graphics container (i.e. the map object), retrieves the envelope of the element and updates the envelope size, and finally calls the shared procedure CreateTiles() in the MappingExtentTool class to recreate tiles.

```

Private Sub UpdateMappingExtent()
    ' first validate extent values
    If Extension.ExtentRight <= Extension.ExtentLeft Or
       Extension.ExtentTop <= Extension.ExtentBottom Then
        Windows.Forms.MessageBox.Show("Invalid mapping extent.",
                                       "Data Driven Pages",
                                       Windows.Forms.MessageBoxButtons.OK,
                                       Windows.Forms.MessageBoxIcon.Error)

        Return
    End If

    ' get the mapping extent element
    Dim extElement As IElement = FindElement("Mapping Extent")
    If extElement Is Nothing Then Return

    ' get the envelope of the element and update its bounds
    Dim rect As IEnvelope = TryCast(extElement.Geometry, IPolygon).Envelope
    rect.XMin = Extension.ExtentLeft
    rect.YMin = Extension.ExtentBottom
    rect.XMax = Extension.ExtentRight
    rect.YMax = Extension.ExtentTop
    extElement.Geometry = rect

    ' call the CreateTiles() procedure in the MappingExtentTool class
    MappingExtentTool.CreateTiles(rect, Extension.TileRows, Extension.TileCols)
End Sub

```

The above procedure calls the FindElement() function to find a graphic element in the graphics container by providing a name of the element to the function. The FindElement() function returns an element object.

```

Private Function FindElement(ElementName As String) As IElement
    Dim gc As IGraphicsContainer = My.ArcMap.Document.FocusMap
    gc.Reset()
    Dim element As IElement = gc.Next()
    Dim props As IElementProperties = Nothing
    While element IsNot Nothing
        props = element
        If props.Name.ToUpper() = ElementName.ToUpper() Then
            Return element
        End If
        element = gc.Next()
    End While
    Return Nothing
End Function

```

After creating the above procedures, you can run the program and test this functionality.

5.2 Resizing the graphic extent element

Since the graphic mapping extent element is placed on the top in the graphics container, the user can use the ArcMap “Select Elements” tool to select it. Once a graphic element is selected, eight handles appears on the corners and edges. We hope that when the user drags the handles to resize this mapping extent element, tiles are resized accordingly and the mapping extent text boxes are also updated.

This functionality needs an event to react to changes made to the graphic element. Because the graphic element is a dynamically added object, VB.NET does not provide event procedure for the element. ArcObjects provide an IGraphicsContainerEvents interface that contains events listening to changes occurring to elements in the graphics container (Figure 5). When an element in the graphics is resized, the ElementUpdated event is fired.

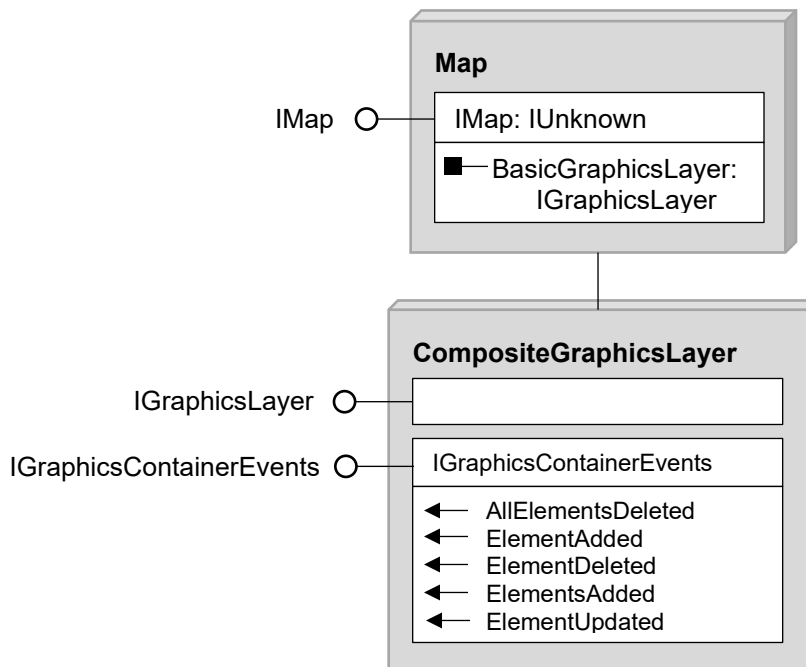


Figure 5 The IGraphicsContainerEvents interface and its events

To create a custom event using the IGraphicsContainerEvents interface involves a few steps. First, you need to get a map object (with IMap interface) from the map document, then get a CompositeGraphicsLayer object with IGraphicsLayer interface from the BasicGraphicsLayer property on the IMap interface. These tasks can be carried out by the following code.

```
Dim mxdoc As IMxDocument = My.ArcMap.Document      ' gets the map document
Dim map As IMap = mxdoc.FocusMap                  ' gets the focus map
Dim glyr As IGraphicsLayer = map.BasicGraphicsLayer ' gets the graphics layer
```

Second, you need to declare a member (class-level) delegate object variable to represent the IGraphicsContainerEvents_Event interface which contains event procedures.

IGraphicsContainerEvents_Event is a hidden interface that actually holds the events of the IGraphicsContainerEvents interface. This hidden interface does not show in IntelliSense in Visual Studio, but you can find it using the Visual Studio object browser (Figure 6).

You can launch the object browser in VB by clicking the main menu View >> Object Browser. Once the browser opens, you can type a class or interface name in the search box and hit Enter key to search. To show hidden types and members, you need to click the “Object Browser Setting” button, and check “Show Hidden Types and Members” (Figure 6).

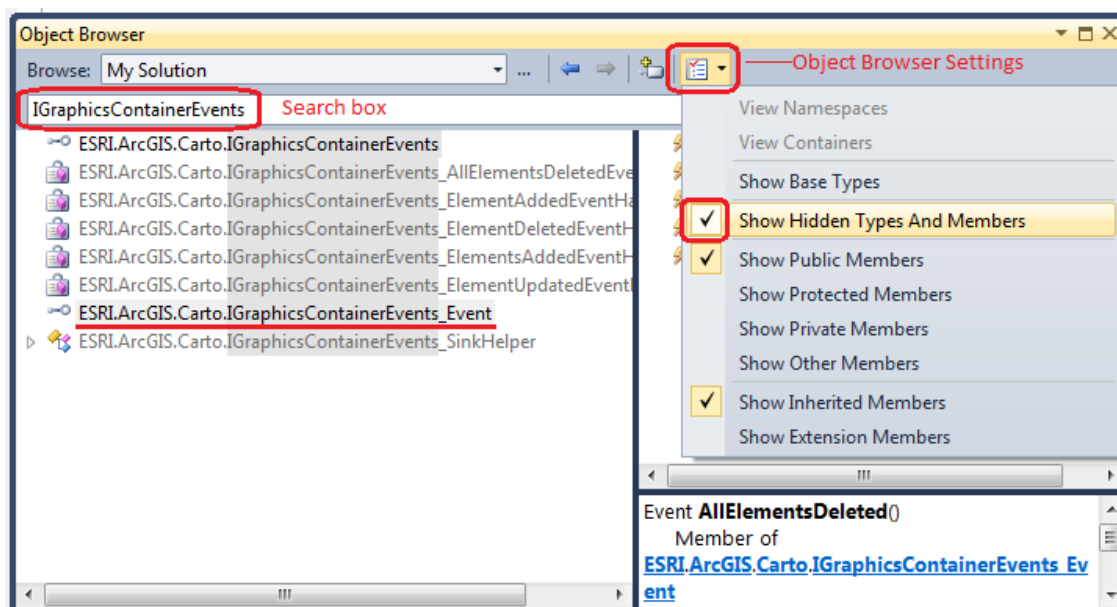


Figure 6 Using the Visual Studio object browser to find object interfaces

The following line declares a member delegate object variable in a class.

```
Private gcEvent As IGraphicsContainerEvents_Event
```

Third, we need to execute command AddHandler to add the address of a custom procedure into a specific event to handle. For example,

```
AddHandler gcEvent.ElementUpdated, AddressOf OnElementUpdated
```

OnElementUpdated is a custom procedure that will be created by you to carry out tasks when event ElementUpdated (gcEvent.ElementUpdated) is fired. The name of the procedure does not matter as long as the procedure performs the intended tasks.

Finally, when an event is no longer needed or when the application closes, a custom event can be removed by command RemoveHandler. For example,

```
RemoveHandler gcEvent.ElementUpdated, AddressOf OnElementUpdated
```

In this project, a reasonable class to implement this event is the Extension class. When ArcMap is started and the Extension class is loaded, the OnStartup event of the class is automatically invoked. Therefore, the OnStartup event of the Extension class is where you will write code to

create the element updated event. When ArcMap closes, the OnShutdown event of the Extension class is fired, therefore you can remove the custom event.

Please follow steps below to create the element updated event.

- 1) Import the following namespaces in the Extension.vb module.

```
Imports ESRI.ArcGIS.Framework
Imports ESRI.ArcGIS.ArcMapUI
Imports ESRI.ArcGIS.Carto
Imports ESRI.ArcGIS.Geometry
Imports ESRI.ArcGIS.esriSystem
```

- 2) Declare a member delegate object variable in the Extension class.

```
Private gcEvent As IGraphicsContainerEvents_Event ' delegate object variable
```

- 3) Implement the OnStartup() procedure as the following.

```
Protected Overrides Sub OnStartup()
    Dim mxdoc As IMxDocument = My.ArcMap.Document ' gets the map document
    Dim map As IMap = mxdoc.FocusMap ' gets the focus map
    Dim glyr As IGraphicsLayer = map.BasicGraphicsLayer ' gets the graphics layer

    ' get the IGraphicsContainerEvents_Event interface
    gcEvent = CType(glyr, IGraphicsContainerEvents_Event)

    ' use the AddHandler command to add the pointer (address)
    ' of a custom procedure into a specified event to handle
    AddHandler gcEvent.ElementUpdated, AddressOf OnElementUpdated
End Sub
```

- 4) Remove the event using the following code in the OnShutdown() event procedure.

```
RemoveHandler gcEvent.ElementUpdated, AddressOf OnElementUpdated
```

- 5) Implement the custom procedure OnElementUpdated(). This procedure updates the map extent text boxes in the dockable window and recreates tile elements.

```
Private Sub OnElementUpdated(element As IElement)
    ' make sure parameter element receives a rectangle element
    If Not TypeOf element Is IRectangleElement Then Return

    ' exit if current view is page layout
    Dim av As IActiveView = My.ArcMap.Document.ActiveView
    If TypeOf av Is IPageLayout Then Return

    ' make sure the element is the mapping extent rectangle
    Dim elementProps As IElementProperties = element
    If elementProps.Name <> "Mapping Extent" Then Return

    ' get the polygon object of the element
    Dim poly As IPolygon = TryCast(element.Geometry, Polygon)
    If poly Is Nothing Then Return
    Dim rect As IEnvelope = poly.Envelope ' gets the envelope object
```

```

' update text boxes in the dockable window
If WindowLoaded Then
    ExtentTop = rect.YMax
    ExtentBottom = rect.YMin
    ExtentLeft = rect.XMin
    ExtentRight = rect.XMax
    DockableWindow.TextBox_ExtTop.Text = Math.Round(rect.YMax, 6)
    DockableWindow.TextBox_ExtBottom.Text = Math.Round(rect.YMin, 6)
    DockableWindow.TextBox_ExtLeft.Text = Math.Round(rect.XMin, 6)
    DockableWindow.TextBox_ExtRight.Text = Math.Round(rect.XMax, 6)
End If

' call the CreateTiles() procedure in MappingExtentTool class to recreate tiles
MappingExtentTool.CreateTiles(rect, TileRows, TileCols)
End Sub

```

Now, you can run the program to test the element updated event. When ArcMap opens, load a layer into the map, use the mapping extent tool to draw a rectangle in the map to define a mapping area. Then use the ArcMap select tool (which will be automatically selected after drawing the extent) to select the overall box and drag the handles around the box to resize it.

6. Creating a new grid index feature class

6.1 Using GxDialog

Before creating a grid index feature class, we must allow the user to specify an output geodatabase, feature dataset, and feature class name. This can be accomplished by launching a GxDialog in the MouseClick event of the output text box (txtOutput). A GxDialog object allows the user to browse to a geodatabase and dataset as well as to specify a feature class name. Once the user provides the information, the program will preserve the destination geodatabase, feature dataset, and feature class names in three member variables of the dockable window class, and display the full path in the output text box.

To implement this functionality, please declare three member variables in the dockable window (DockableWindow) class as the following.

```

Private gdbName As String      ' geodatabase name
Private fdsName As String      ' feature dataset name
Private fclassName As String   ' feature class name

```

Then implement the MouseClick event of the output text box (txtOutput) as the following.

```

Private Sub txtOutput_MouseClick(...) Handles txtOutput.MouseClick
    Dim saveDialog As IGxDialog = New GxDialog ' instantiates a GxDialog object
    saveDialog.Title = "Save feature class"    ' sets a title

    ' set geodatabase object filters
    saveDialog.ObjectFilter = New GxFilterFGDBFeatureClasses ' file gdb filter
    saveDialog.ObjectFilter = New GxFilterPGDBFeatureClasses ' personal gdb filter

    If Not saveDialog.DoModalSave(0) Then ' Cancel button clicked
        txtOutput.Clear()
    End If
End Sub

```

```

        btnSave.Enabled = False
        Return
    End If

    ' check if a feature dataset is browsed
    If saveDialog.FinalLocation.Category.IndexOf("Feature Dataset") > 0 Then
        ' a feature dataset is browsed
        gdbName = saveDialog.FinalLocation.Parent.FullName ' gets geodatabase name
        fdsName = saveDialog.FinalLocation.BaseName        ' gets dataset name
    Else ' no feature dataset involved
        gdbName = saveDialog.FinalLocation.FullName        ' gets geodatabase name
        fdsName = ""
    End If
    fclassName = saveDialog.Name ' gets feature class name
    saveDialog = Nothing        ' dispose the GxDialog object

    ' display the gdb, dataset, and feature class in the text box
    txtOutput.Text = gdbName & "\"
    If fdsName <> "" Then txtOutput.Text &= fdsName & "\"
    txtOutput.Text &= fclassName
    btnSave.Enabled = True ' enable the Save button
End Sub

```

6.2 Saving tile features in feature class

Since each tile element has a polygon (rectangle) geometry and each has been assigned a name, we can create a new polygon feature based on a tile element and save the feature in a feature class. These tasks can be carried out in the Click event procedure of the Save Result button.

Saving tiles to a new feature class involves four major tasks.

- 1) creating a blank polygon feature class
- 2) getting all tile elements from the graphics container
- 3) creating tile features based on tile elements and save features in the feature class
- 4) displaying the new feature class in the focus map

The first three tasks need to use methods of the GDBManager class introduced in Section 3.1 of Chapter 13. Please load the GDBManager.vb module (in folder Source_Code on CD-ROM) into the current project.

The first task involves a number of data management operations. First, you need to check existence of the user provided feature class in the specified geodatabase and properly handle existing index feature class; second, you need to create a blank polygon feature class; and third, you need to create user selected fields, PageName, PageNumber, and UTM zone field, in the new feature class. This task is implemented by the following function.

```

Private Function CreateGridIndexFeatureClass(map As IMap) As IFeatureClass
    ' 1) check existence of feature class with the specified name
    Dim gdbmgr As New GDBManager() ' instantiates a GDBManager object
    Dim fclass As IFeatureClass = gdbmgr.GetFeatureClass(gdbName, fclassName)

    If fclass IsNot Nothing Then
        If chkOverwrite.Checked Then
            ' delete the feature class

```

```

        gdbmgr.DeleteFeatureClass(gdbName, fclassName, fdsName)
    Else
        MsgBox.Show("Feature class already exists in the geodatabase.",
                    "Data Driven Pages",
                    MsgBoxButtons.OK, MsgBoxIcon.Exclamation)
        Return Nothing
    End If
End If

' 2) create a new feature class
If fdsName <> "" Then
    fclass = gdbmgr.CreateFeatureClass(gdbName, fdsName, fclassName,
                                       esriGeometryType.esriGeometryPolygon)
Else
    fclass = gdbmgr.CreateFeatureClass(gdbName, fclassName,
                                       esriGeometryType.esriGeometryPolygon,
                                       Map.SpatialReference)
End If

If fclass Is Nothing Then
    MsgBox.Show("Creating new index feature class failed.",
                "Data Driven Pages",
                MsgBoxButtons.OK, MsgBoxIcon.Error)
    Return Nothing
End If

' 3) create user specified fields
' a. create PageName field
If chkCreatePageName.Checked Then
    Dim field As IField = gdbmgr.CreateField("PageName",
                                             esriFieldType.esriFieldTypeString, False, False, , 32)
    fclass.AddField(field)
End If

' b. create PageNumber field
If chkCreatePageNumber.Checked Then
    Dim field As IField = gdbmgr.CreateField("PageNumber",
                                             esriFieldType.esriFieldTypeInteger, False, False)
    fclass.AddField(field)
End If

' c. create the UTM_Zone field
If chkCalcUTMZones.Checked Then
    Dim field As IField = gdbmgr.CreateField("UTM_Zone",
                                             esriFieldType.esriFieldTypeString, False, False, , 600)
    fclass.AddField(field)
End If
Return fclass
End Function

```

The second task can be accomplished by the following GetTileElements() function. The function takes a map object (with IGraphicsContainer interface) as input and returns an array list object that contains all tile elements retrieved from the map.

```

Private Function GetTileElements(gc As IGraphicsContainer) As ArrayList
    gc.Reset() ' reset the pointer in the graphics container
    Dim elem As IElement = gc.Next() ' gets the first element

```

```

Dim prop As IElementProperties

Dim Tiles As New ArrayList      ' a container used to hold tile elements
While elem IsNot Nothing
    prop = elem                ' QI to IElementProperties interface
    If prop.Name.Substring(0, 5) = "Tile:" Then ' a tile found
        Tiles.Insert(0, elem) ' adds a tile into the arraylist
    End If
    elem = gc.Next()           ' gets next element
End While
Return Tiles
End Function

```

ArrayList is a Visual Studio built-in collection that can be used to hold any type of objects. Inside the While repetition, the If block checks if an element is a tile whose name starts with letters "Tile:" (see Section 4). If an element is a tile, insert it into the array list collection. To place an object on top in the collection, argument 0 is passed to the z-order index parameter of the Insert() method.

Creating features and saving them into a feature class are introduced in Chapter 13. The third task can be carried by the following sub procedure (SaveTiles()). This procedure requires a feature class object and an array list of tile elements as input. From each tile element in the array list, a polygon object is retrieved and a polygon feature is created based on it. The PageName field of the feature is set by deriving a page name from the tile element, the PageNumber field is set with a serial number, and the UTM_Zone field is set by calling a function that creates a factory code for spatial reference.

```

Private Sub SaveTiles(FeatureClass As IFeatureClass, Tiles As ArrayList)
    ' validate input
    If FeatureClass Is Nothing Then Return
    If Tiles Is Nothing Then Return
    If Tiles.Count = 0 Then Return

    ' get indexes of necessary fields
    Dim fcursor As IFeatureCursor = FeatureClass.Insert(False)
    Dim xPName As Long = fcursor.FindField("PageName")
    Dim xPNum As Long = fcursor.FindField("PageNumber")
    Dim xUTM As Long = fcursor.FindField("UTM_Zone")

    ' declare some necessary variables
    Dim fBuffer As IFeatureBuffer = Nothing
    Dim poly As IPolygon = Nothing
    Dim props As IElementProperties = Nothing
    Dim counter As Integer = 1

    ' create a polygon feature for each tile and insert it into the feature class
    For Each tile As IElement In Tiles
        poly = TryCast(tile.Geometry, IPolygon) ' get element geometry
        fBuffer = FeatureClass.CreateFeatureBuffer() ' creates a new feature
        fBuffer.Shape = poly ' sets feature shape
        props = tile ' QI to IElementProperties

        If xPName > 0 Then ' recall tile name format is "Tile: A#"
            ' extract the row letter and column # from tile name as page name
            fBuffer.Value(xPName) = props.Name.Substring(6) ' sets page name field
        End If
    Next
End Sub

```

```

    If xPNum > 0 Then ' sets page number field
        fBuffer.Value(xPNum) = counter
        counter += 1
    End If

    If xUTM > 0 Then ' sets UTM zone field
        fBuffer.Value(xUTM) = CreateUTM(tile)
    End If

    fcursor.InsertFeature(fBuffer) ' inserts feature into cursor
Next
fcursor.Flush()
End Sub

```

Given a tile element, the following function creates a spatial reference represented by a UTM coordinate system factory code. The function first makes a copy of the geometry of the input tile element, gets its spatial reference, and projects the spatial reference to a Geographic Coordinate System (GCS). With a GCS, a central meridian of the tile element can be derived, on which a UTM zone is calculated. With a UTM zone code, a spatial reference can be created by calling the MakeSpatialReferencePRJ() method of the GDBManager class. Finally, the function returns the factory code to the client.

```

Private Function CreateUTM(tile As IElement) As String
    ' get tile spatial reference
    Dim tileSpref As ISpatialReference = tile.Geometry.SpatialReference
    ' get tile envelope
    Dim tileRect As IEnvelope = tile.Geometry.Envelope

    ' make a copy of the tile envelope
    Dim rect As IEnvelope = New Envelope
    With rect
        .SpatialReference = tileSpref
        .XMin = tileRect.XMin
        .XMax = tileRect.XMax
        .YMin = tileRect.YMin
        .YMax = tileRect.YMax
    End With

    Dim gdbmgr As New GDBManager() ' instantiate a GDBManager object

    ' create a new GCS_WGS1984 spatial reference object
    Dim spref As ISpatialReference = gdbmgr.MakeSpatialReferenceGCS(
        esriSRGeoCSType.esriSRGeoCS_WGS1984)
    rect.Project(spref) ' reproject the tile copy to GCS_WGS1984

    Dim cm As Double = (rect.XMin + rect.XMax) / 2 ' central meridian of the tile
    Dim zone As Integer = Math.Floor(180 + cm) / 6 + 1 ' calculates a UTM zone

    ' create a spatial reference object from a zone code
    ' Note: with NAD, UTM zone factory codes are "269" + zone code
    Dim zonecode As String = "269"
    If zone < 10 Then
        zonecode &= "0" & zone
    Else
        zonecode &= zone
    End If

```

```

    Return zonecode      ' return the factory code
End Function

```

The last task is to display a new feature class in a map. It can be carried out by the following procedures. The only new skill used in this procedure is to set transparency to a layer. This is accomplished by setting the Transparency property on the ILayerEffects interface of the feature layer object. This procedure calls a function to create a simple fill renderer object. Function CreateSimpleFillRenderer() is also provided below.

```

Private Sub DisplayFeatureClass(FeatureClass As IFeatureClass , map As IMap)
    ' First, clear all elements in the graphics container
    Dim gc As IGraphicsContainer = map
    gc.DeleteAllElements()      ' all graphics elements are no longer needed

    ' Second, create a new feature layer
    Dim flyr As IGeoFeatureLayer = New FeatureLayer
    flyr.FeatureClass = FeatureClass

    Dim lyr As ILayer = flyr
    lyr.Name = FeatureClass.AliasName

    ' create and set layer renderer
    Dim fcolor As IColor = CreateColor(96, 225, 128)
    Dim bcolor As IColor = CreateColor(255, 0, 255)
    Dim rndr As IFeatureRenderer = CreateSimpleFillRenderer(fcolor, bcolor, 2)
    flyr.Renderer = rndr

    ' QI to ILayerEffects interface to set layer transparency
    Dim lyrEffects As ILayerEffects = flyr
    lyrEffects.Transparency = 50

    map.AddLayer(lyr)
    Dim av As IActiveView = map
    av.Refresh()
End Sub

Private Function CreateSimpleFillRenderer(ByVal fillColor As IColor,
                                         ByVal borderColor As IColor,
                                         ByVal borderWidth As Double) _
                                         As ISimpleRenderer
    ' instantiate a simple renderer object
    Dim srndr As ISimpleRenderer = New SimpleRenderer()

    Dim fsymbol As IFillSymbol = New SimpleFillSymbol() ' creates fill symbol
    fsymbol.Color = fillColor      ' sets fillcolor of the fill symbol

    Dim lsymbol As ISimpleLineSymbol = New SimpleLineSymbol() ' creates a line symbol
    lsymbol.Color = borderColor    ' sets border color
    lsymbol.Width = borderWidth    ' sets border width
    fsymbol.Outline = lsymbol      ' sets the outline property of the fill symbol

    srndr.Symbol = fsymbol         ' sets the symbol property

    Return srndr      ' return the renderer object
End Function

```

Once procedures to carry out tasks are created, the Click event of the Save Result button can be implemented as the following.

```
Private Sub btnSave_Click(...) Handles btnSave.Click
    ' check geodatabase name and feature class name
    If gdbName = "" Or fclassName = "" Then
        MessageBox.Show("Output location is not specified.", "Data Driven Mapping",
            Windows.Forms.MessageBoxButtons.OK,
            Windows.Forms.MessageBoxIcon.Warning)

        Return
    End If

    ' get the map objects
    Dim map As IMap = My.ArcMap.Document.FocusMap

    ' first, create a blank polygon feature class in the geodatabase
    Dim fclass As IFeatureClass = CreateGridIndexFeatureClass(map)
    If fclass Is Nothing Then Return

    ' second, get tile elements from the graphics container
    Dim gc As IGraphicsContainer = map ' QI
    Dim Tiles As ArrayList = GetTileElements(gc)
    If Tiles.Count = 0 Then
        MessageBox.Show("Mapping extent is not specified.",
            "Data Driven Mapping",
            Windows.Forms.MessageBoxButtons.OK,
            Windows.Forms.MessageBoxIcon.Warning)

        Return
    End If

    ' third, create and save tile features in the new feature class
    SaveTiles(fclass, Tiles)

    ' fourth, display the new feature class as several layers
    DisplayFeatureClass(fclass, map)
End Sub
```

Now, you can run the program and test the saving index feature class functionality. To do so,

- 1) Load some layers and create a map.
- 2) Use the mapping extent tool to define an extent.
- 3) Edit the tile of rows and columns (you must hit the Enter key after editing a number).
- 4) Use the ArcMap "Select Elements" tool to adjust the extent to your satisfaction, and delete any unnecessary (blank) tiles.
- 5) Delete tiles that you do not want to map. To do so, select a tile and hit the Delete key.
- 6) You may delete the overall extent element if no further adjustment will be made.
- 7) Click the output text box to select an output file or personal geodatabase, and provide a feature class name.
- 8) Click the Save Result button to finish.

If everything goes well, a grid index layer should be created and appear in the map as a semitransparent layer.

7. Setting up Data Driven Pages

Once a map is loaded and a grid index layer is created or loaded (make sure the index layer is on top), we are ready to set up data driven pages. We want the “Set Up Data Driven Pages” button to accomplish the following tasks.

- 1) Switching to the page layout view and properly position the map frame on the layout page (Figure 7).
- 2) Inserting a static map title into the layout page.
- 3) Inserting a dynamic page name element in the layout page if the user selects the option.
- 4) Inserting a dynamic scale bar element in the layout page if the user selects the option.
- 5) Inserting a dynamic page number element in the layout page if the user selects the option.
- 6) Bringing up the ArcGIS built-in Data Driven Pages toolbar.
- 7) Open the ArcGIS built-in Data Driven Pages setup dialog box (Figure 2).

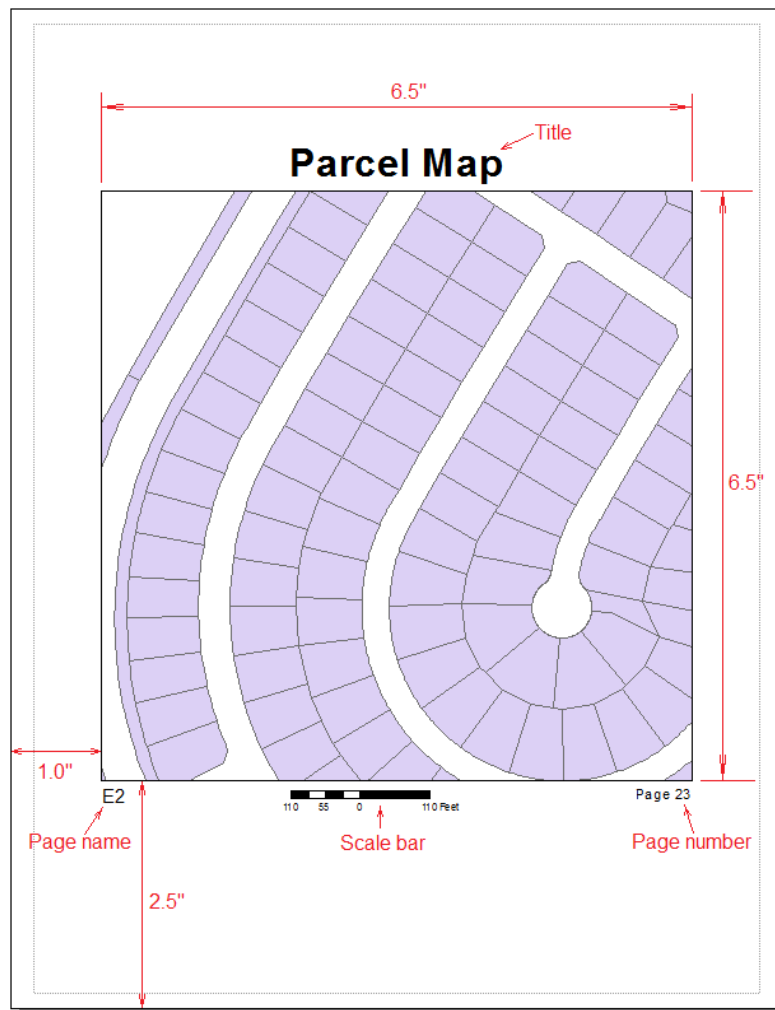


Figure 7 Page layout design

Tasks 6 and 7 are simple since you just need to find out the GUID (see Section 4) of the two ArcGIS built-in commands and call the GetCommand() function to get the commands. You

have create a GetCommand() function in the MappintExtentTool class. You can either modify that function by replacing the domain keyword “Private” with keywords “Friend Shared” or copy it into the DockableWindows class.

For the first five tasks, you will create five procedures each carrying out a task. Sub procedure SetupMapFrame() will carry out the first task. It first switches the map to the page layout view and positions the map frame element. To position the map frame, the program gets the map frame element in the page layout, creates a new envelope object that defines a desired size and position for the map frame, and sets the envelope to the map frame element.

```
Private Sub SetupMapFrame()
    Dim mxdoc As IMxDocument = My.ArcMap.Document      ' gets the document
    Dim map As IMap = mxdoc.FocusMap                  ' gets the focus map
    My.ArcMap.Document.ActiveView = mxdoc.PageLayout ' switches to page layout view

    ' resize and position the map frame in the page layout view
    Dim gc As IGraphicsContainer = mxdoc.PageLayout ' QI
    Dim elem As IElement = gc.FindFrame(map)        ' gets the map frame element

    ' create a new envelope object for the map frame element
    Dim rect As IEnvelope = New Envelope()
    rect.PutCoords(1, 2.5, 7.5, 9.0) ' arguments: xmin, ymin, xmax, ymax (inches)
    elem.Geometry = rect              ' sets the envelope to the map frame element
End Sub
```

The second task is performed by sub procedure InsertTitle(). This procedure creates a text element and positions it at top center of the map frame. To position the text element, it gets an envelope object from the map frame element, then creates a point object by referring to the top-center location of the envelope, and sets the point object to the new text element.

```
Private Sub InsertTitle()
    If txtMapTitle.Text = "" Then Return ' no map title provided by the user

    ' create a text element object
    Dim txt As ITextElement = New TextElement() ' instantiates an object
    txt.Text = txtMapTitle.Text                 ' sets the Text property
    Dim sym As ITextSymbol = New TextSymbol ' creates a text symbol object
    sym.Font = GetIFontDisp("Arial", 32, Drawing.FontStyle.Bold) ' sets font
    txt.Symbol = sym                            ' sets the symbol to the text element

    ' get the envelope of the map frame
    Dim gc As IGraphicsContainer = My.ArcMap.Document.PageLayout
    Dim map As IMap = My.ArcMap.Document.FocusMap
    Dim mapFrame As IElement = gc.FindFrame(map) ' gets the map frame element
    Dim poly As IPolygon = mapFrame.Geometry    ' gets the geometry
    Dim rect As IEnvelope = poly.Envelope       ' gets the envelope of the map frame

    ' create a point object by referring to the top center of the envelop
    Dim p As IPoint = New Point()
    p.PutCoords((rect.XMin + rect.XMax) / 2, rect.YMax + 0.15) ' 0.15" above the top

    Dim elem As IElement = txt ' QI to IElement interface of the text element
    elem.Geometry = p          ' positions the text element object
    gc.AddElement(elem, 0)     ' adds the text element into the graphics container
End Sub
```

The InsertTitle() procedure calls the following function to create a font object.

```
Private Function GetIFontDisp(FontFamily As String, FontSize As Single,
                             FontStyle As System.Drawing.FontStyle) _
    As stdole.IFontDisp
    Dim font As New System.Drawing.Font(FontFamily, FontSize, FontStyle)
    Return TryCast(ESRI.ArcGIS.ADF.COMSupport.OLE.GetIFontDispFromFont(font),
                  stdole.IFontDisp)
End Function
```

To insert a dynamic page name in the page layout for the third task, you have to get a built-in Data Driven Pages command. You can find the ArcMap GUID of the insert page name command at the ArcGIS Resources website. Once a page number text element is inserted, you can assign a name to the element and increase its font size and relocate it to the lower-left corner (by default, it is located at the center of the map). These tasks are carried out by the following procedure.

```
Private Sub InsertPageName()
    ' call the insert page name ArcGIS command
    Dim insPagename As ICommandItem = _
        GetCommand("{316BE38A-419E-4B73-A300-E4635D560D1F}")
    insPagename.Execute() ' execute the command

    ' get the inserted page name text element (must be on the top of all layers)
    Dim gc As IGraphicsContainer = My.ArcMap.Document.PageLayout
    gc.Reset()
    Dim elem As IElement = gc.Next ' gets the latest inserted element
    Dim props As IElementProperties = elem ' QI
    props.Name = "PageName Element" ' sets a name to it in case necessary

    ' change the font-size of the element
    Dim txtelem As ITextElement = elem ' QI
    Dim sym As ITextSymbol = New TextSymbol ' creates a new text symbol
    sym.Size = 16 ' sets font-size
    txtelem.Symbol = sym ' sets the symbol to the text element

    ' position the element at the lower-left corner of the map frame
    ' first, get the envelope of the map frame
    Dim mapFrame As IElement = gc.FindFrame(My.ArcMap.Document.FocusMap)
    Dim poly As IPolygon = mapFrame.Geometry
    Dim rect As IEnvelope = poly.Envelope

    ' then create a point object by referring to the lower-left corner.
    Dim p As IPoint = New Point
    p.PutCoords(rect.XMin + 0.15, rect.YMin - 0.25)
    elem.Geometry = p ' sets the point to the element to position it
End Sub
```

To perform the fourth task that inserts a page number dynamic element into the page layout is very similar to the third task except that you need to use a different built-in Data Driven Pages command. Since we want this text element to be located near the lower right corner, you will also need to adjust the position in relation to the map frame. This task can be implemented by the following InsertPageNumber() procedure.

```

Private Sub InsertPageNumber()
    ' call the insert page name ArcGIS command
    Dim insPagnum As ICommandItem = _
        GetCommand("{316BE38B-419E-4B73-A300-E4635D560D1F}")
    insPagnum.Execute()

    Dim gc As IGraphicsContainer = My.ArcMap.Document.PageLayout
    gc.Reset()

    ' get the page name text element which must be on the top
    Dim elem As IElement = gc.Next
    Dim props As IElementProperties = elem
    props.Name = "PageNum Element" ' sets a name to the element

    Dim txtelem As ITextElement = elem
    Dim sym As ITextSymbol = New TextSymbol
    sym.Size = 12
    txtelem.Symbol = sym
    txtelem.Text = "Page " & txtelem.Text ' adds word "Page" ahead

    ' get the envelope of the map frame
    Dim mapFrame As IElement = gc.FindFrame(My.ArcMap.Document.FocusMap)
    Dim poly As IPolygon = mapFrame.Geometry
    Dim rect As IEnvelope = poly.Envelope

    ' create a point object
    Dim p As IPoint = New Point
    p.PutCoords(rect.XMax - 0.3, rect.YMin - 0.2) ' lower-right corner
    elem.Geometry = p
End Sub

```

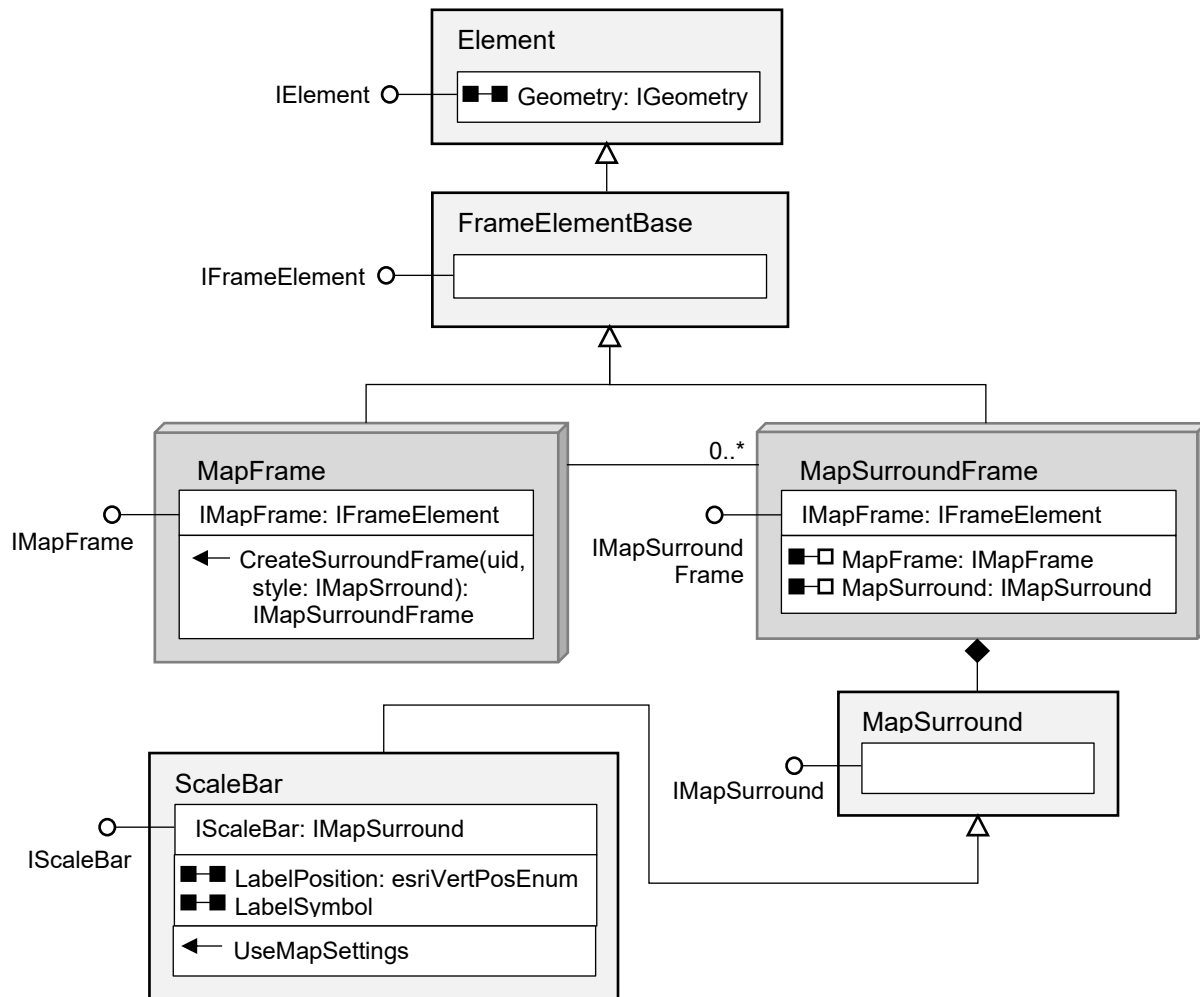


Figure 8 Object model diagram for scale bar

For the fourth task that inserts a scale bar into the page layout, you must understand map surrounds. Map surrounds are specific types of elements that are associated with a Map object. A scale bar is a map surround since it is an element associated with a map. When the scale of a map changes, the scale bar is automatically updated. Other examples of map surrounds include legend and north arrows.

All map surrounds are held in a **MapSurroundFrame** container which is also an element, and this element is related to a **MapFrame**. Figure 8 shows that given the UID object of a map surround object, a **MapSurroundFrame** object (with **IMapSurroundFrame** interface) can be obtained from a **MapFrame** object by calling its **CreateSurroundFrame()** method. If you pass the UID of a scale bar element to the **CreateSurroundFrame()** method, the resulting **MapSurroundFrame** element holds a scale bar. To specify a size and position to the new scale bar, you can set an envelope object to the **Geometry** property on the **IElement** interface (inherited) of the **MapSurroundFrame** element.

If you want to customize the scale bar element such as to change the text symbol of labels, relative position of labels, color of the bar, and units, you can get the **MapSurround** object (**IMapSurround** interface) from the **MapSurroundFrame** element by calling its **MapSurround**

property. Because you know this MapSurround object is a ScaleBar object (you get it with a scale bar UID), you can cast it to a ScaleBar object. The IScaleBar interface has a number of properties that allow you to customize the scale bar object. Method UseMapSettings() on the IScaleBar interface can usually be called to set division and units of the scale bar based on the current map. The following procedure accomplishes the task to insert a scale bar into the page layout.

```
Private Sub InsertScalebar()
    Dim map As IMap = My.ArcMap.Document.FocusMap
    Dim PageLayout As IPageLayout = My.ArcMap.Document.PageLayout

    ' get the envelope of the map frame
    Dim gc As IGraphicsContainer = PageLayout
    Dim mapElement As IElement = gc.FindFrame(map)
    Dim poly As IPolygon = mapElement.Geometry
    Dim rect As IEnvelope = poly.Envelope ' envelope of the map frame
    Dim xc As Double = (rect.XMin + rect.XMax) / 2 ' x coordinate of the center

    ' create an envelope object by referring to the lower center of the map frame
    ' the envelope will be used define the location and size of the scale bar
    Dim envelope As IEnvelope = New Envelope
    envelope.PutCoords(xc - 1.25, rect.YMin - 0.35,
        xc + 1.25, rect.YMin - 0.1)

    ' Create a scale bar Surround
    ' first, create a UID object for the scale bar
    Dim uid As ESRI.ArcGIS.esriSystem.UID = New ESRI.ArcGIS.esriSystem.UID
    uid.Value = "esriCarto.AlternatingScaleBar"
    ' second, create a map surround frame object
    Dim mapFrame As IMapFrame = TryCast(mapElement, IMapFrame) ' QI
    Dim mapSurroundFrame As IMapSurroundFrame =
        mapFrame.CreateSurroundFrame(uid, Nothing) ' Dynamic Cast
    ' third, set the envelope to the map surround frame
    Dim element As IElement = TryCast(mapSurroundFrame, IElement) ' QI
    element.Geometry = envelope ' sets an envelope to the scale bar element

    Dim mapSurround As IMapSurround = mapSurroundFrame.MapSurround
    Dim markerScaleBar As IScaleBar = CType(mapSurround, IScaleBar) ' Explicit Cast
    markerScaleBar.LabelPosition = ESRI.ArcGIS.Carto.esriVertPosEnum.esriBelow
    markerScaleBar.UseMapSettings()

    Dim av As IActiveView = PageLayout
    element.Activate(av.ScreenDisplay) ' activate the scale bar element
    gc.AddElement(element, 0)
End Sub
```

Once all procedures are created, you can easily implement the “Set Up Data Driven Pages” button as the following.

```
Private Sub btnSetupDDP_Click(...) Handles btnSetupDDP.Click
    If My.ArcMap.Document.FocusMap.LayerCount = 0 Then
        MessageBox.Show("You must load necessary layers into the map",
            "Data Driven Pages",
            MessageBoxButtons.OK, MessageBoxIcon.Exclamation)
    Return
    End If
```

```

SetupMapFrame()      ' sets up the map frame in the page layout
InsertTitle()        ' inserts a static map title
If chkShowPageName.Checked Then InsertPageName() ' inserts page name
If chkShowScalebar.Checked Then InsertScalebar() ' inserts scale bar
If chkShowPageNumber.Checked Then InsertPageNumber() ' inserts page number

' turn off display of the index layer (top layer)
My.ArcMap.Document.FocusMap.Layer(0).Visible = False
My.ArcMap.Document.ActiveView.Refresh()

' turn on the built-in data driven pages toolbar
Dim ddpToolBar As ICommandBar = _
    GetCommand("{EF545F90-9F94-462E-8692-035923C87193}")
ddpToolBar.Dock(esriDockFlags.esriDockShow)

' display the built-in DDP setup dialog
Dim ddpSetup As ICommandItem = _
    GetCommand("{6F19555B-2C72-4DB4-83BC-F74DDA21E274}")
ddpSetup.Execute()
End Sub

```

To test this button, you need to load a data layer and then create a grid index layer or load an index layer on top of the focus map. When you click the button, the page layout view should be activated, the map frame, map title, page name, scale bar, and page number elements should all be properly positioned in the page (Figure 7). Also the built-in Data Driven Pages toolbar should be brought up and the Set up Data Driven Pages dialog box must be popped up for you to enable Data Driven Pages. After enabling and properly setting DDP, you can use the built-in DDP toolbar to view each page.

Summary and Highlights

Data Driven Pages are a series of layout pages created from different sections of a map using one layout design. The extent of each page is defined by a feature in an index layer. ArcGIS provides a set of tools for creating index layers and setting up Data Driven Pages.

This chapter demonstrates the process of using ArcObjects and .NET programming to create custom Data Driven Pages add-in tools that facilitate creation of a grid index feature class, the design of page layout, and configuration of Data Driven Pages. The process involves an array of ArcObjects and add-in programming techniques including using the rubber band object to track geometric shapes, creating and displaying graphics elements, creating new feature classes in a geodatabase, creating new features and inserting them into a feature class, sharing information between classes, using add-in extension, creating custom events, etc. The following highlights a few new techniques introduced in this chapter.

To share a member variable or procedure of a class to peer classes, you can declare the member with keywords “Friend Shared”. A shared variable or procedure can be accessed from a peer class by referring it with the host class name as a namespace without instantiating the host class. To share a control (e.g. text box) on a form (window), you need to declare a shared delegate member variable in the form class and assign the control to the delegate variable.

An add-in extension is a special class in an add-in project. It is used to coordinate activities between other add-in components such as buttons, tools and dockable windows in a project. You can use an extension to host shared variables and procedures in a project so that all add-in component can access them.

ArcObjects provide interfaces that contain events reacting to some user interactions in ArcGIS Desktop environment. For example, the IDocumentEvents interface of the MxDocument component provides events to respond to actions such as active view changed, document opened, new document created, and documents closed. Interface IGraphicsContainerEvents contains a numbers of events that react to element changes in a graphics container. The ElementUpdated event is invoked when any element in a graphics container is changed.

To create a custom event involves a few steps. First, you need to get the component that hosts the interface containing the relevant event; second, you need to declare a member delegate variable to represent the interface; third, you need to execute command AddHandler to add the pointer (i.e. address) of a custom procedure into a specific event for it to handle; finally, you may need to remove the event by executing the RemoveHandler command when a custom event is no longer needed.

Elements in a graphics container do not automatically have an identifier. The developer can assign a name to an element by using the Name property on the IElementProperties. To find a specific element in a graphics container, all elements in the container has to be examined by their names.

Exercises

1. Answer the following questions.
 - 1) What are Data Driven Pages?
 - 2) What is an index layer for creating Data Driven Pages?
 - 3) Once you have a map and a grid index layer, how do you set up Data Driven Pages using ArcGIS built-in tools?
 - 4) What are map surrounds?
 - 5) How do you write code to display a built-in toolbar in an add-in project?
 - 6) What is an add-in extension?
 - 7) How do you write code to expose a text box in a dockable window to peer classes in an add-in project?
 - 8) How do you create a TextChanged event procedure that reacts to several text boxes?
 - 9) In a KeyDown event, how do you know if the key pushed is the Enter key?
 - 10) How do you use code to assign a name to a graphics element?
 - 11) How do you find a specific graphics element in a graphics container given a name?
 - 12) How do you create a custom event that reacts to changes to a graphic element in a graphics container?
 - 13) How do you create a custom event that responds to the deletion of a graphic element from a graphics container?
 - 14) What is the GxDialog component? How do you use it?
 - 15) How do you change the transparency of a layer in a map document?
 - 16) How do you change the size and position of a map frame in a layout page?
 - 17) How do you position a text element at a given coordinate?
 - 18) How do you insert a page name into a layout page enabled with Data Driven Pages?
 - 19) How do you insert a scale bar element into a layout page?
 - 20)
2. Create a new ArcMap add-in project that contains an extension. In the extension create a custom event that is fired when a new documents is created, and a custom event that is fired when a document is open. In the procedures that is handled by these events, you only need to send a message box to signal that the event is fired.